

代码清单递归版的记忆化

下面是一份可以直接运行的代码。先用朴素的递归实现再展示记忆化版本。两者恰好构成指数复杂度到线性复杂度的对照。代码行保持原始缩进。中文注释行与代码行同处一个清单。

```
// naive-recursive.ts
export function fibNaive(n: number): number {
  递归终止条件时直接返回n
  if (n <= 1) return n;
  return fibNaive(n - 1) + fibNaive(n - 2);
}

// memoized.ts
export function fibMemo(n: number, memo = new Map<number, number>()): number {
  if (n <= 1) return n;
  const hit = memo.get(n);
  if (hit !== undefined) return hit;
  const value = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
  memo.set(n, value);
  return value;
}
```

朴素版本对每个重复计算子问题调用树按指数增长。记忆化版本把已算出的答案存进缓存。每个只真正计算一次。二者代码只差几行，性能却差着数量级。这正是递归缓存组合的经典教材案例。

值得注意的细节：作为默认参数挂在函数签名里，调用方不需要知道缓存的存在。在 TypeScript 的类型标注同时，约束了键与值的类型。把实现换成数组缓存或 Map 版本，接口保持不变。

复杂度对照

```
fibNaive(35)  // ~ 150 ms    O(2^n)  exponential
fibMemo(35)   // < 1 ms    O(n)     linear
fibNaive(20)  // ~ 1 ms    still exponential
fibMemo(500)  // < 2 ms    linear, fits in double
```

测试时建议把 `n` 参数控制在 35 以内，否则等待时间会明显干扰断言与超时设置。记忆化版本则可以轻松算到 500，往上就需要防止精度丢失。

第二条清单：调用栈观察器

调试递归时，缩进日志比断点单步高效得多。下面的帮助函数在进入与返回时各打印

一行缩进随深度变化肉眼即可看到调用树的结构。

```
export function traced<T>(label: string, depth: number, run: () => T): T {
  const pad = "  ".repeat(depth);
  console.log(`${pad}-> ${label}`);
  const result = run();
  console.log(`${pad}<- ${label} = ${String(result)}`);
  return result;
}
// traced("fib(4)", 0, () => fibNaive(4));
```

把套在外层即可观察到整棵调用树套在内层则会看到树被

缓存压扁的过程同样的观察器也能用在目录遍历与树遍历上是排查递归逻辑的第一工具本清单到此收束。